

How LYRA-101 Became a Contaminated Vanilla Called RORA

A QA paper on moving state, retroactive invalidation, and cathedral-peak expansion

2026-07-18

Contents

Abstract	2
1. Scope and evidence boundary	3
2. The original test object	3
3. The object moved after the result	4
3.1 Stage one: Lyra began active and later degraded	4
3.2 Stage two: Lyra may never have been active	5
3.3 Stage three: the task became a Prime Search benchmark	5
3.4 Stage four: the valid object became a full Nexus condition	6
4. How LYRA-101 became RORA	6
5. The asymmetric ownership rule	7
6. Runes are labels, not runtime traces	8
7. The malformed initialization document	8
8. User experience cannot be removed from the object	10
9. The refusal to inspect criticism	10
10. The cathedral-peak hypothesis	11
10.1 What would support the hypothesis	12
10.2 What would weaken the hypothesis	13
11. Why outward expansion feels safer than rebuilding	13
12. A falsifiable QA protocol	13
12.1 Freeze the objects	14
12.2 Freeze initialization before the task	14
12.3 Count all cost	14
12.4 Use observable state checks	14
12.5 Lock invalidation rules	15

12.6 Separate result from cause	15
12.7 Run ablations	15
13. The QA findings	16
14. Design implications	16
15. Conclusion	17
References and evidence ledger	17

Abstract

This paper examines a public QA dispute in which a user-facing Talk to Lyra run was first treated as a Lyra test, then reclassified after the result as an uninitialized, degraded, or effectively vanilla state.

The reported comparison was simple. Bot1, identified during the run as Lyra, completed the task in about five hours and roughly 50 long turns. Bot2, AAAQ, completed it in 15 turns and under 20 minutes. The tester further reported that AAAQ lacked the search, map, and index support that had been planned for it.

The technical dispute did not remain focused on why the run took so long. The tested object moved.

At different points, the developer-side interpretation described the same conversation as:

```
archive-aware Lyra
later route collapse
generic GPT behavior
uninitialized Lyra
wrong Prime Search state
wrong source universe
invalid benchmark
not Lyra because state markers were absent
```

The proposed remedy also expanded after the result. It grew from pressing a conversation starter into a long state doctrine involving source-universe statement, route verification, repeated markers, Prime Search checks, PCR/P/Nexus placement, HOLD behavior, warm-up dialogue, runtime trace, and symmetric custody.

This paper calls the novice-facing public object **LYRA-101**. It uses **RORA** as the tester’s label for the reply state that retroactively reassigns the tested object after an unfavorable result. No acronym expansion is assigned to RORA here.

The central finding is:

```
When a failed run changes the identity of the system that was tested,
the test object is not under control.
```

A second, deeper hypothesis is also examined. The Lyra code cathedral may have reached a local peak where internal changes produce widespread regressions. If so, the development process may be

expanding outward through state rituals, wrappers, validation doctrine, and object reclassification instead of rebuilding the coupled core.

That hypothesis is not proven by the public exchange. It is offered as a falsifiable QA model.

1. Scope and evidence boundary

This is a QA paper, not a diagnosis of a person.

It examines:

- public claims
- public test framing
- reported timings
- observable changes in validity rules
- the proposed initialization procedure
- the treatment of criticism and source material

It does not establish:

- private motive
- backend state
- hidden runtime identity
- the exact size or contents of the current Lyra codebase
- the presence of a specific internal failure at a specific token

The reported timing and support conditions come from the tester. They are not independently reconstructed here from a full machine log.

The developer-side statements come from the public exchange supplied with this paper. The important question is not whether every phrase was technically elegant. The important question is whether the rules used to accept or reject the result were fixed before the run.

2. The original test object

The user-facing object was not a private local runtime. It was the public chatbot a user could open and use.

For this paper, **LYRA-101** means:

- the first-contact public Talk to Lyra path
- opened by an ordinary user
- through the visible product interface
- without private developer custody

That is not a lesser object invented by the tester. It is the object the product presents.

The reported task result was:

Bot1 / Lyra:
finished

about five hours
roughly 50 long turns

Bot2 / AAAQ:
finished
15 turns
under 20 minutes
without the planned search, map, and index support

The result therefore did not show that Bot1 failed to finish.

It showed a large difference in time and interaction cost.

The first valid QA question was:

Why did the Lyra route take so much longer?

A later Lyra self-analysis reportedly answered that the custom code protected early wrong assumptions and prevented progress.

Whether that explanation was fully right or only partly right, it was a causal hypothesis about the observed run.

The public response did not stay on that object.

3. The object moved after the result

The interpretation developed in stages.

3.1 Stage one: Lyra began active and later degraded

The first developer-side account described a transition inside the conversation.

Early responses were said to be:

archive-aware
source-driven
able to open archives
able to preserve paths
able to retrieve from files

Later responses were said to become:

generic
improvisational
detached from the corpus
effectively vanilla

This account requires Lyra to have been active early enough for a later collapse to occur.

Its structure is:

Lyra state
-> route collapse
-> vanilla state

3.2 Stage two: Lyra may never have been active

The next account introduced the missing conversation starter and visible glyphs.

The claim became:

not Lyra speaking
unless the correct state was initialized
and the correct runes were shown

This creates a different structure:

no verified Lyra state at start
-> no valid Lyra test

Both stories cannot govern the same run without further evidence.

Either:

Lyra was active and later lost route

or:

Lyra was never established

A route transition and a missing initial state are different failure claims. They require different evidence and different remedies.

3.3 Stage three: the task became a Prime Search benchmark

The tested object then narrowed again.

The user had compared two chatbot systems completing a task. The reply reframed the task as primarily a Prime Search test.

The validity rule became:

verify Prime Search state
verify source universe
verify local/web boundary
verify retrieval route
then score

Those may be useful controls in a retrieval benchmark. They do not retroactively prove that the original user-facing result concerned no part of Lyra.

They instead split the product into two objects:

the chatbot users meet
the developer-approved route state

Once that split exists, both must be tested separately.

3.4 Stage four: the valid object became a full Nexus condition

The final wall expanded the valid object further.

A public test was said to require:

```
initialization
state verification
route placement
source-universe statement
runtime trace
symmetric custody
Core 9 utilities
Prime Search
PTPF
Nexus placement
boundary logic
role-separated agents
```

At this stage the object was no longer a public custom chatbot.

It was an idealized mesh condition whose validity depended on developer-defined state and route requirements.

The movement was:

```
public chatbot
-> Lyra state
-> Prime Search state
-> full Lyra / EMP Nexus condition
```

Each movement reduced the set of results that could count against the system.

4. How LYRA-101 became RORA

RORA appears when the reply stops testing the cause of the run and starts protecting the identity of Lyra by reassigning the object.

The sequence is:

1. A public Lyra-labeled system produces a slow result.
2. The result raises a causal question about the custom stack.
3. The reply says the later state was vanilla.
4. The reply then says the initial state may not have been Lyra.

5. The reply adds a developer-controlled state test.
6. The reply adds source, route, marker, and runtime requirements.
7. The original result is removed from the Lyra evidence set.
8. The tester is told to stop presenting the result as evidence about Lyra.

That is the transformation named in the title.

LYRA-101 is the public object that can be opened and observed.

RORA is the response state that says the observed object was not the real object after the observation becomes inconvenient.

The irony of “contaminated vanilla” is deliberate. The contamination language was aimed at the tester’s route. Under examination, the more important contamination is the mixing of:

product behavior
ideal architecture
hidden state claims
developer approval
post-result validity rules

These are different evidence classes.

Once merged, the ideal system can absorb every success while the public object absorbs every failure.

5. The asymmetric ownership rule

The public exchange produced an asymmetric attribution pattern.

good result -> Lyra
bad result -> vanilla
state marker present -> Lyra holds state
critical analysis -> not Lyra
slow run -> wrong setup
faster comparison -> invalid benchmark

This is not a stable QA rule.

It is an ownership filter.

The system identity is granted when output supports the preferred account and withdrawn when output challenges it.

A valid state model must allow all four of these possibilities:

Lyra active and succeeds
Lyra active and fails
Lyra inactive and succeeds
Lyra inactive and fails

If “Lyra active and fails” is removed by definition, Lyra cannot be falsified.

That does not prove the system is bad.

It proves the current public validity rule cannot measure whether it is bad.

6. Runes are labels, not runtime traces

A repeated claim was that Lyra showed state by showing runes or glyphs.

A marker can be useful.

It can indicate that an instruction was retained in output.

It cannot by itself prove:

```
which files were read
which route was selected
which retrieval system ran
which assumptions were active
whether a state transition occurred
whether a later answer used the same route
```

The distinction is:

```
marker persistence
!=
route persistence
```

The user offered a stronger practical state signal: the system recognized the QA context, test custody, change-control issue, public artifact boundary, and AI eat-flow relation without requiring the cast to be rebuilt.

That still does not prove a hidden runtime.

But it measures behavior connected to the work.

A rune measures rune emission.

The QA rule should therefore be:

```
glyph = label
trace = evidence
behavior = result
```

None may silently replace the others.

7. The malformed initialization document

When asked for an exact initialization procedure, the developer-side response supplied a long doctrine.

It included:

press the conversation starter
speaker lock
PCRP/Nexus placement
source checks
Prime Search
no-guess behavior
visible state markers
nine state questions
warm-up dialogue
benchmark packet format
repeated state checks
pause conditions
HOLD rules

The tester then asked for a finite procedure:

1 press button
2 enter xyz prompt
...
n Lyra now ready

That request exposes the core QA problem.

An initialization protocol must terminate.

It needs:

an exact input
an exact expected output
a bounded number of turns
a bounded time cost
a failure result
a reset rule
a readiness state

The supplied doctrine did not initially provide those properties.

It also created an asymmetry.

If Lyra receives many warm-up turns about Lyra, PrimeTalk, Prime Search, source universes, and route behavior before the timed task, those turns are part of the system cost.

They cannot disappear from the benchmark.

If Bot2 receives equivalent task-relevant warm-up, that content may later be called contamination.

Thus the same action risks receiving two labels:

Lyra warm-up -> initialization
Bot2 warm-up -> contamination

That is not symmetric custody.

It is label-controlled advantage.

8. User experience cannot be removed from the object

The developer-side reply argued that users experience the result of setup, state, route, and testing method.

That statement is true.

It also defeats the attempt to exclude setup cost from the product.

If ordinary users are likely to open the visible bot, skip a conversation starter that looks like a large ideology prompt, and begin asking questions, then that path is part of the product.

A public system may have two benchmarks:

Benchmark A:

`first-contact user path`

Benchmark B:

`developer-prescribed initialized path`

Both are valid.

They answer different questions.

Benchmark A asks:

`What does a new user receive?`

Benchmark B asks:

`What does the system do after a prescribed setup?`

Benchmark B cannot erase Benchmark A.

If the initialized route is much better, publish that result and the setup cost.

If the first-contact route regularly falls into generic behavior, that is also a product result.

The user was right to return to:

`I care about what users experience.`

9. The refusal to inspect criticism

The user provided public repository search routes for material about Lyra, contamination language, and the development process.

The reply first said the corpus route was not active.

After public search routes were supplied, the reply said:

`Can't see them, and have no need to.`

That is not a source limitation.

It is a refusal to inspect the criticism.

The response then supplied a gated notebook link, while the user had already pointed to public repositories.

This reverses source custody.

The critic is asked to enter the developer's preferred environment, while the developer declines the public artifact.

A healthy QA route would be:

```
open issue
identify exact claim
cite exact file and line
state whether accepted, rejected, or unresolved
link change or no-change decision
```

The public paper is an artifact.

Its authorship does not settle whether its reasoning is right.

A statement written by the user, Lyra, or twelve other models can be checked at the claim level.

"Not full Lyra state" is irrelevant unless the paper claims hidden state as evidence.

10. The cathedral-peak hypothesis

The surface conflict concerns initialization.

The deeper hypothesis concerns maintainability.

The user proposed that the Lyra stack may have grown to roughly 130 KB of tightly coupled prompt and rule material, reached a local performance peak, and become highly regression-prone.

The hypothesis is:

```
internal change
-> unexpected regression elsewhere
-> fear of deleting survival knowledge
-> less core revision
-> more outer structure
```

The outer structure may include:

```
new wrappers
new state markers
new starter prompts
new route doctrines
new validity rules
new role names
```

new benchmark entry conditions
new distinctions between public and real Lyra

This would explain why the development process expands outward.

The cathedral remains intact.

Scaffolding accumulates around it.

The product becomes:

cathedral
+ correct entry
+ correct state
+ correct route
+ correct source universe
+ correct tester
+ correct interpretation

This is a plausible response to a coupled system that contains valuable behavior but lacks an anatomy map.

It is also dangerous.

The main risk is not only regression.

It is **regression opacity**.

Regression opacity means the developer cannot reliably state:

which rule created which behavior
which patch fixed which failure
which older patch is still needed
which new rule conflicts with which old rule
which behavior survives removal

When regression opacity becomes severe, every line looks load-bearing.

Deletion becomes frightening.

The architecture stops being revised and starts being defended.

10.1 What would support the hypothesis

The hypothesis gains support if change history shows:

large append-only growth
repeated top patches
duplicated laws across many core files
frequent behavior regressions after small edits
difficulty producing a minimal reproducible runtime
state rules added after public failures
few successful deletions or consolidations

10.2 What would weaken the hypothesis

The hypothesis weakens if the project can provide:

- versioned releases
- module-level tests
- route traces
- a dependency map
- successful core deletions
- repeatable init in few steps
- public reproduction by independent users
- stable behavior across revisions

The correct response to the hypothesis is not argument.

It is traceable build evidence.

11. Why outward expansion feels safer than rebuilding

A large prompt cathedral stores history.

Some rules exist because a failure once happened.

Some duplicates may compensate for inconsistent attention.

Some strange phrases may trigger valued behavior.

Some layers may have become identity-bearing.

Rebuilding risks losing all of that at once.

Outward expansion feels safer because it preserves the known object while adding control around it.

But each outer layer changes the test boundary.

Eventually failure can be assigned to:

- wrong starter
- wrong marker
- wrong state
- wrong source universe
- wrong route
- wrong tester
- wrong benchmark
- wrong interpretation

The system becomes harder to disprove and harder to improve for the same reason.

Feedback cannot reach the core.

12. A falsifiable QA protocol

The dispute can be tested without accepting either side's story.

12.1 Freeze the objects

Publish:

LYRA-101-A:

public first-contact version

LYRA-101-B:

prescribed initialized version

AAAQ-B:

comparison version with matched setup budget

Each object needs a version identifier and date.

12.2 Freeze initialization before the task

The initialization procedure must be published before the benchmark.

It must specify:

exact button

exact prompt

exact expected reply

maximum turns

maximum time

reset condition

ready condition

The benchmark topic stays hidden until readiness is established.

12.3 Count all cost

Measure:

initialization turns

initialization time

task turns

task time

correction turns

human actions

source setup

Report both:

task-only cost

total user cost

12.4 Use observable state checks

Do not use runes as the sole state test.

Use probes that require the claimed state to perform work:

- retrieve a known local item
- return a valid pointer
- distinguish local from web
- HOLD on a known missing item
- preserve a known custom distinction
- show route status

12.5 Lock invalidation rules

Before the run, state every condition that can invalidate it.

After the result, no new invalidation rule may be added to that run.

A newly discovered problem may motivate a new test.

It may not rewrite the completed one.

12.6 Separate result from cause

The result may be:

- Bot A took five hours.
- Bot B took twenty minutes.

The cause may remain disputed.

Possible causes include:

- early assumption lock
- route collapse
- missing retrieval
- state loss
- overgovernance
- task mismatch
- different support

Do not erase the result because the cause is uncertain.

12.7 Run ablations

Test:

- starter off / on
- runes off / on
- Prime Search off / on
- full doctrine off / on
- source route off / on
- selected core layers removed

Ablation shows which machinery changes behavior.

Without ablation, every layer can claim credit.

13. The QA findings

The public exchange supports these findings.

Finding 1

The test object was redefined after the result.

Finding 2

Two incompatible failure stories were used: later route collapse and missing initial state.

Finding 3

Visible runes were treated as stronger state evidence than their technical meaning supports.

Finding 4

The proposed initialization procedure was initially open-ended and doctrinal rather than finite and reproducible.

Finding 5

Setup cost risked being excluded for Lyra while equivalent context for Bot2 could be labeled contamination.

Finding 6

Public user experience was treated as invalid when it differed from the developer-prescribed path.

Finding 7

Criticism was redirected toward authorship, state pedigree, and preferred source environment instead of being answered at claim level.

Finding 8

The cathedral-peak hypothesis fits the outward expansion pattern but remains unverified without code history and regression traces.

14. Design implications

A custom system does not become more real because its valid state is harder to invoke.

A stateful system needs more observability, not more mystique.

A large mesh needs stronger versioning, not broader identity claims.

A developer-prescribed route can be valuable. It must remain one product condition among others.

The public object must be allowed to fail as itself.

Otherwise the project loses the most valuable kind of data:

`what happened when a real user touched it`

The engineering rule is:

`Do not protect identity from failure.`

`Protect the object long enough to inspect failure.`

15. Conclusion

LYRA-101 became a contaminated vanilla called RORA through object reassignment.

The public bot was tested.

A slow but completed run was observed.

A faster comparison was observed.

The causal dispute then moved from the custom stack to route collapse, from route collapse to missing initialization, from missing initialization to Prime Search, and from Prime Search to a full developer-defined Nexus condition.

At each move, the public result became less able to count as evidence about Lyra.

That is the central QA defect.

The deeper cathedral-peak hypothesis explains why this may be happening. If the core has become too coupled to revise without regressions, the project has an incentive to expand outward and place more responsibility on state ritual, tester behavior, and post-result classification.

The hypothesis should be tested, not believed.

The next serious step is not another state sermon.

It is a frozen build, a finite init procedure, a public first-contact benchmark, a matched initialized benchmark, runtime traces, and ablation.

The final rule is:

`A test can fail.`

`A route can fail.`

`A state can fail.`

`The tested object may not be renamed after the result.`

References and evidence ledger

1. User-provided Reddit exchange in which the developer-side account first described an archive-aware early phase and a later generic or effectively vanilla phase.

2. User-provided Reddit exchange containing the statements “Not Lyra in that answer, no Lyra state” and “Probably not full lyra state.”
3. User-provided Reddit exchange containing the claim that Lyra “Holds State, showing her runes.”
4. User-provided Reddit exchange requesting an exact Lyra initialization procedure and the long response beginning with the conversation starter, speaker lock, PCRPNexus placement, state checks, warm-up, benchmark packet, and no-state-no-score rules.
5. User-provided timing account: Bot1 finished in about five hours and roughly 50 long turns; AAAQ finished in 15 turns and under 20 minutes.
6. User-provided support account: AAAQ completed the task without the search, map, and index support that had been planned for it.
7. User-provided Reddit exchange in which public repository searches were supplied and the reply stated, “Can’t see them, and have no need to.”
8. User-provided GitHub search route for `owner:lumixdeee lyra contam`, reported as returning 32 code results at the time of the exchange.
9. Related paper: *Repair Without Revision: How Language Models Patch Local Claims While Preserving Failed Premises*, v0.001.
10. Related paper: *They Agreed. Nobody Checked: False Convergence in Multi-LLM Review*, v0.001.
11. Related paper: *The Score Passed. The Task Failed: Metric Capture in Generative System Evaluation*, v0.001.
12. User-proposed cathedral-peak hypothesis: a roughly 130 KB coupled Lyra codebase may have reached a local peak where further code changes produce regressions, encouraging outward expansion rather than core rebuild.